

Competitive Programming

Section 32 Editorial: Combinatorics Under Modulo

Competitive Programming Course

2026-07-18

Table of contents

1	Editorial overview	4
2	Shared tool: factorials under a prime modulus	5
3	A. Choose Queries	6
3.1	Restatement and constraints	6
3.2	Observations and walk-through	6
3.3	Pseudocode	6
3.4	Correctness proof	6
3.5	Complexity	6
3.6	Implementation notes and wrong answers	7
3.7	Full C++ solution	7
3.8	Full Python solution	7
4	B. Rearrange Letters	9
4.1	Restatement and constraints	9
4.2	Observations and walk-through	9
4.3	Pseudocode	9
4.4	Correctness proof	9
4.5	Complexity	9
4.6	Implementation notes and wrong answers	10
4.7	Full C++ solution	10
4.8	Full Python solution	10
5	C. Distribute Candies	12
5.1	Restatement and constraints	12
5.2	Observations and walk-through	12
5.3	Pseudocode	12
5.4	Correctness proof	12
5.5	Complexity	12
5.6	Implementation notes and wrong answers	12
5.7	Full C++ solution	13
5.8	Full Python solution	13
6	D. Separated Lineup	15
6.1	Restatement	15
6.2	Constraints and traps	15
6.3	Observations	15
6.4	Sample walk-through	16
6.5	Algorithm	16
6.6	Correctness proof	17
6.7	Complexity	17
6.8	C++ implementation notes	17
6.9	Python implementation notes	18
6.10	Full C++ solution	18

6.11 Full Python solution	19
6.12 Common wrong answers	20
6.13 Tests that matter	20
7 Testing and stress discussion	21
8 What to carry forward	22

1 Editorial overview

Problem	Mathematical object	Required factorial limit
A. Choose Queries	$C(n, k)$	maximum n
B. Rearrange Letters	$n! / \text{product}(c!)$	string length
C. Distribute Candies	$C(n+s-1, n-1)$	maximum $n+s-1$
D. Separated Lineup	choose gaps, then order distinct people	maximum $b+1$

2 Shared tool: factorials under a prime modulus

All local factorial indices are below $\text{MOD}=1,000,000,007$; this is a required precondition, not a cosmetic fact. It ensures $i!$ has an inverse modulo MOD . The local maximum index is at most two million, so the technique is valid.

Store arrays indexed from 0 through `limit` with the invariant:

```
fact[i]      = i! mod MOD
invfact[i]   = inverse(i!) mod MOD
```

```
fact[0] = 1
for i = 1..limit: fact[i] = fact[i-1] * i
invfact[limit] = fact[limit]^(MOD-2)
for i = limit down to 1: invfact[i-1] = invfact[i] * i
C(n,k) = 0 if k < 0 or k > n, otherwise fact[n]*invfact[k]*invfact[n-k]
```

The backward recurrence follows from $i! = i \cdot (i-1)!$. It costs $O(\text{limit})$ time, one $O(\log \text{MOD})$ exponentiation, and $O(\text{limit})$ memory; $C(n,k)$ is then $O(1)$. In C++, multiply as `long long`; in Python use `pow(x, MOD-2, MOD)`.

3 A. Choose Queries

3.1 Restatement and constraints

Answer up to 200,000 values of $C(n,k)$ for $0 \leq k \leq n \leq 10^6$. Computing the product from scratch per query is too slow, and ordinary division after a modulo is invalid. Read all queries first because the largest n determines the one shared preprocessing limit.

3.2 Observations and walk-through

For $C(6,2)$, ordinary arithmetic says $6!/(2!4!)=15$. The code instead multiplies three residues:

```
fact[6] * invfact[2] * invfact[4] mod MOD = 15
```

For $C(10,0)$ and $C(10,10)$, the same formula uses $\text{fact}[0]=\text{invfact}[0]=1$, so both answers are one without branches.

3.3 Pseudocode

```
read all (n,k) queries
prepare(maximum n)
for each (n,k): print C(n,k)
```

3.4 Correctness proof

The shared invariant gives $\text{fact}[n]=n!$ and inverse factorials for both denominator terms. Since all indices are below prime MOD , those denominator factorials are invertible. Their product with $\text{fact}[n]$ is exactly $n!/(k!(n-k)!)$ in modular arithmetic, which is $C(n,k)$. $\square$

3.5 Complexity

```
Preprocessing:  $O(\max n)$  time and memory
Each query:     $O(1)$  time
```

3.6 Implementation notes and wrong answers

Do not precompute `invfact[i]` with an exponentiation inside a loop. Do not omit `fact[0]=1`. A generic combination helper should reject invalid `k` even though this local input guarantees a valid range.

3.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;

long long power(long long a, long long e) {
    long long r = 1;
    while (e) { if (e & 1) r = r * a % MOD; a = a * a % MOD; e >>= 1; }
    return r;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int q; if (!(cin >> q)) return 0;
    vector<pair<int,int>> queries(q); int limit = 0;
    for (auto &[n,k] : queries) { cin >> n >> k; limit = max(limit, n); }
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; ++i) fact[i] = fact[i-1] * i % MOD;
    invfact[limit] = power(fact[limit], MOD - 2);
    for (int i = limit; i >= 1; --i) invfact[i-1] = invfact[i] * i % MOD;
    for (auto [n,k] : queries)
        cout << fact[n] * invfact[k] % MOD * invfact[n-k] % MOD << '\n';
}
```

3.8 Full Python solution

```
import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)
    for i in range(limit, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact
```

```

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    queries = [(data[i], data[i + 1]) for i in range(1, 2 * q + 1, 2)]
    fact, invfact = prepare(max(n for n, _ in queries))
    print("\n".join(str(fact[n] * invfact[k] % MOD * invfact[n-k] % MOD)
                     for n, k in queries))

if __name__ == "__main__":
    main()

```

4 B. Rearrange Letters

4.1 Restatement and constraints

Given one lowercase string of length at most 10^6 , count its distinct rearrangements modulo MOD. The trap is that equal letters are not distinct objects, so a direct $n!$ overcounts.

4.2 Observations and walk-through

For aabc, pretend the two as are labelled a_1, a_2 . There are $4!=24$ labelled permutations, but a_1abc and a_2abc are the same visible string. Every visible arrangement is counted $2!$ times, so the answer is $24/2=12$. For aabbc, two independent duplicate groups give $5!/(2!*2!)=30$.

4.3 Pseudocode

```
count every letter
prepare(length of string)
answer = fact[length]
for each letter count c:
    answer = answer * invfact[c] mod MOD
print answer
```

4.4 Correctness proof

Label all equal copies temporarily. There are $n!$ labelled orders. For a fixed visible arrangement, the copies of each character can be permuted in $c!$ ways without changing it; independent character groups multiply these counts. Therefore each visible arrangement has exactly $\text{product}(c!)$ labelled preimages, and multiplying by inverse factorials computes the quotient. $\square$

4.5 Complexity

```
Time:     $O(n + \text{alphabet size})$ 
Memory:  $O(n)$ 
```

4.6 Implementation notes and wrong answers

Use a 26-entry frequency array; counts of absent characters are zero and `invfact[0]=1`, so they are harmless. Recomputing `inverse(fact[c])` separately for every letter is acceptable for 26 letters but hides the reusable engine. Do not count $n!$ when the string has duplicates.

4.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;
long long power(long long a, long long e) {
    long long r = 1;
    while (e) { if (e & 1) r = r*a % MOD; a = a*a % MOD; e >>= 1; }
    return r;
}
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    string s; if (!(cin >> s)) return 0;
    int n = (int)s.size();
    vector<long long> fact(n + 1, 1), invfact(n + 1, 1);
    for (int i = 1; i <= n; ++i) fact[i] = fact[i-1] * i % MOD;
    invfact[n] = power(fact[n], MOD - 2);
    for (int i = n; i >= 1; --i) invfact[i-1] = invfact[i] * i % MOD;
    int count[26]{};
    for (char ch : s) ++count[ch - 'a'];
    long long answer = fact[n];
    for (int c : count) answer = answer * invfact[c] % MOD;
    cout << answer << '\n';
}
```

4.8 Full Python solution

```
import sys

MOD = 1_000_000_007

def main() -> None:
    s = sys.stdin.buffer.read().strip()
    if not s:
        return
    n = len(s)
    fact = [1] * (n + 1)
    for i in range(1, n + 1):
        fact[i] = fact[i - 1] * i % MOD
```

```

invfact = [1] * (n + 1)
invfact[n] = pow(fact[n], MOD - 2, MOD)
for i in range(n, 0, -1):
    invfact[i - 1] = invfact[i] * i % MOD
counts = [0] * 26
for ch in s:
    counts[ch - ord('a')] += 1
answer = fact[n]
for c in counts:
    answer = answer * invfact[c] % MOD
print(answer)

if __name__ == "__main__":
    main()

```

5 C. Distribute Candies

5.1 Restatement and constraints

For each of up to 200,000 queries, distribute s identical candies among n distinct children, allowing zero. Both are at most 10^6 , so the largest factorial index is $n+s-1$, at most 1,999,999, still strictly below MOD. Precomputing merely to maximum n is a frequent wrong answer.

5.2 Observations and walk-through

The equation is $x_1 + \dots + x_n = s$ with nonnegative variables. For $n=3, s=4$, the string `**|**` represents (2,1,1). There are four stars and two bars; choose two bar positions among six, giving $C(6,2)=15$. For $n=1, s=10$, choose zero bars, $C(10,0)=1$.

5.3 Pseudocode

```
read all (n,s) queries
prepare(maximum over queries of n+s-1)
for each query: print C(n+s-1, n-1)
```

5.4 Correctness proof

Map a distribution to a row containing x_1 stars, a bar, x_2 stars, and so on. This mapping is reversible: the stars in each gap recover the unique distribution. There are $s+n-1$ positions and selecting the $n-1$ bar positions therefore counts all and only valid distributions. $\square$

5.5 Complexity

```
Preprocessing: O(L) time and memory, L = max(n+s-1)
Each query:    O(1) time
```

5.6 Implementation notes and wrong answers

Read all queries before allocating so the maximum uses $n+s-1$. Children are distinct but candies are identical: use bars, not a permutation count. The formula already handles $s=0$ and $n=1$; extra branches are unnecessary.

5.7 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1000000007LL;
long long power(long long a, long long e) {
    long long r = 1;
    while (e) { if (e & 1) r = r*a % MOD; a = a*a % MOD; e >>= 1; }
    return r;
}
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int q; if (!(cin >> q)) return 0;
    vector<pair<int,int>> queries(q); int limit = 0;
    for (auto &[n,s] : queries) { cin >> n >> s; limit = max(limit, n+s-1); }
    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; ++i) fact[i] = fact[i-1] * i % MOD;
    invfact[limit] = power(fact[limit], MOD - 2);
    for (int i = limit; i >= 1; --i) invfact[i-1] = invfact[i] * i % MOD;
    for (auto [n,s] : queries) {
        int all = n + s - 1, bars = n - 1;
        cout << fact[all] * invfact[bars] % MOD * invfact[all-bars] % MOD << '\n';
    }
}
```

5.8 Full Python solution

```
import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)
    for i in range(limit, 0, -1):
        invfact[i - 1] = invfact[i] * i % MOD
    return fact, invfact

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
```

```

queries = [(data[i], data[i + 1]) for i in range(1, 2 * q + 1, 2)]
fact, invfact = prepare(max(n + s - 1 for n, s in queries))
out = []
for n, s in queries:
    all_positions = n + s - 1
    bars = n - 1
    out.append(str(fact[all_positions] * invfact[bars] % MOD
                  * invfact[all_positions - bars] % MOD))
print("\n".join(out))

if __name__ == "__main__":
    main()

```

6 D. Separated Lineup

6.1 Restatement

There are r distinct red performers and b distinct blue performers. Count the full lineups in which every performer appears exactly once and no adjacent pair is red-red. The answer is required modulo 1,000,000,007 for up to 200,000 queries.

This is the section's blind exercise. The statement describes the restriction but deliberately does not state how to turn it into a binomial coefficient.

6.2 Constraints and traps

r and b are each at most one million. Any one query can have far more than 10^6 valid lineups, so enumeration is impossible and ordinary division is not valid after taking a modulus.

The performers are **distinct** even when they share a color. Swapping R1 and R2 produces a different lineup. It is therefore not enough to count color patterns.

When $r > b+1$, no valid lineup exists: even if every blue performer separates two red performers, there are only $b+1$ places in which to put one red. That case must produce zero before indexing a binomial coefficient with an invalid lower argument.

The largest factorial index used is $b+1$, at most 1,000,001, safely below the prime modulus. Precomputing to $r+b$ is unnecessary and can double the memory used by the factorial arrays.

6.3 Observations

First place only the blue performers. They can be ordered in $b!$ ways. Once a blue order is fixed, it creates exactly $b+1$ visible slots:

```
_ B _ B _ B _
```

At most one red performer may occupy a slot, otherwise two reds would become adjacent. Choose which r of the $b+1$ slots receive a red in:

```
C(b+1, r)
```

ways. Finally, arrange the r distinct red performers in their selected slots in $r!$ ways. The independent choices multiply, giving:

```
answer = b! * C(b+1, r) * r!
```

when $r \leq b+1$, and zero otherwise.

The useful discipline is the order of construction: reserve separators first, then choose legal gaps, then assign labels. It prevents both accidental red-red adjacency and accidental treatment of same-colored performers as identical.

6.4 Sample walk-through

For $r=2$, $b=2$, first arrange the two blue performers. There are $2! = 2$ orders. Consider one of them:

```
_ B1 _ B2 _
```

There are three slots and we choose two, so $C(3,2)=3$ slot choices. For each choice, the two red performers can be assigned in $2!=2$ orders. The total is:

```
2! * C(3,2) * 2! = 2 * 3 * 2 = 12.
```

For $r=3, b=1$, the blue performer gives only two slots:

```
_ B1 _
```

Three reds cannot occupy distinct slots, so the answer is zero.

6.5 Algorithm

```
read all (r,b) queries
limit = maximum over queries of max(r, b+1)
precompute fact[0..limit] and invfact[0..limit]

for each (r,b):
    if r > b+1:
        print 0
    else:
        slots = C(b+1, r)
        print fact[b] * slots * fact[r] modulo MOD
```

The $\max(r, b+1)$ limit also covers malformed-but-guarded impossible queries: the arrays remain valid even before the $r > b+1$ test.

6.6 Correctness proof

We prove that the algorithm returns the number of valid lineups for one query.

Lemma 1. After a blue-performer order is fixed, every valid lineup places at most one red performer in each of the $b+1$ slots around that order.

Proof. The slots are exactly the positions before the first blue, between successive blues, and after the last blue. Two reds placed in one slot would have no blue performer between them and would be adjacent, contradicting validity. Conversely, reds in distinct slots always have at least one blue performer between consecutive selected slots, so they are not adjacent. $\square$

Lemma 2. For a fixed blue order, the number of valid completions is $C(b+1, r) * r!$ when $r \leq b+1$, and zero otherwise.

Proof. By Lemma 1, a valid completion chooses exactly r distinct slots from $b+1$, which can be done in $C(b+1, r)$ ways. The r red performers are distinct, so each selected slot set has $r!$ label assignments. The mapping from a slot set and assignment to a lineup is reversible. If $r > b+1$, no such slot set exists. $\square$

Theorem. The algorithm prints exactly the number of valid lineups.

Proof. There are $b!$ possible orders of the distinct blue performers. For each such order, Lemma 2 counts all and only its valid completions. The blue order is uniquely recoverable from any completed lineup, so completions from different blue orders do not overlap. Multiplying $b!$ by the count from Lemma 2 produces exactly every valid lineup once. The algorithm computes this product modulo MOD, as requested. $\square$

6.7 Complexity

Let $L = \max \max(r, b+1)$ over all queries.

```
Preprocessing: O(L) time, O(L) memory
Each query:   O(1) time
```

The one modular exponentiation used to make the inverse-factorial table costs $O(\log \text{MOD})$.

6.8 C++ implementation notes

Use `long long` for every modular product. Each factor is below MOD, and the product of two such values is below about 10^{18} , which fits in signed 64-bit. The C++ reference reads every query before allocating, then computes the inverse factorial table by one Fermat inverse and a backward pass.

The formula has three factors. Reduce after each multiplication:

```
answer = fact[blue] * slots % MOD * fact[red] % MOD;
```

6.9 Python implementation notes

Use `pow(fact[limit], MOD - 2, MOD)` for the one modular inverse. Keep output in a list of strings and write it once; repeated printing is unnecessarily slow for the largest query count.

The value `blue + 1` is an ordinary Python integer, but it still must be included in the preprocessing limit before indexing the factorial tables.

6.10 Full C++ solution

```
#include <bits/stdc++.h>
using namespace std;

const long long MOD = 1'000'000'007LL;

long long mod_pow(long long base, long long exponent) {
    long long result = 1;
    while (exponent > 0) {
        if (exponent & 1LL) result = result * base % MOD;
        base = base * base % MOD;
        exponent >>= 1LL;
    }
    return result;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int q;
    if (!(cin >> q)) return 0;
    vector<pair<int, int>> queries(q);
    int limit = 0;
    for (auto& [red, blue] : queries) {
        cin >> red >> blue;
        limit = max(limit, max(red, blue + 1));
    }

    vector<long long> fact(limit + 1, 1), invfact(limit + 1, 1);
    for (int i = 1; i <= limit; ++i) fact[i] = fact[i - 1] * i % MOD;
    invfact[limit] = mod_pow(fact[limit], MOD - 2);
    for (int i = limit; i >= 1; --i) invfact[i - 1] = invfact[i] * i % MOD;

    for (auto [red, blue] : queries) {
        if (red > blue + 1) {
            cout << 0 << '\n';
            continue;
        }
        long long choose_slots = fact[blue + 1] * invfact[red] % MOD
    }
```

```

        * invfact[blue + 1 - red] % MOD;
    long long answer = choose_slots * fact[red] % MOD * fact[blue] % MOD;
    cout << answer << '\n';
}
return 0;
}

```

6.11 Full Python solution

```

import sys

MOD = 1_000_000_007

def prepare(limit: int) -> tuple[list[int], list[int]]:
    fact = [1] * (limit + 1)
    for value in range(1, limit + 1):
        fact[value] = fact[value - 1] * value % MOD
    invfact = [1] * (limit + 1)
    invfact[limit] = pow(fact[limit], MOD - 2, MOD)
    for value in range(limit, 0, -1):
        invfact[value - 1] = invfact[value] * value % MOD
    return fact, invfact

def main() -> None:
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    q = data[0]
    queries = [(data[index], data[index + 1]) for index in range(1, 2 * q + 1, 2)]
    limit = max(max(red, blue + 1) for red, blue in queries)
    fact, invfact = prepare(limit)

    output: list[str] = []
    for red, blue in queries:
        if red > blue + 1:
            output.append("0")
            continue
        slots = fact[blue + 1] * invfact[red] % MOD * invfact[blue + 1 - red] % MOD
        output.append(str(slots * fact[red] % MOD * fact[blue] % MOD))
    sys.stdout.write("\n".join(output) + "\n")

if __name__ == "__main__":
    main()

```

6.12 Common wrong answers

Counting only color patterns and forgetting the $r!$ and $b!$ label permutations.
Using $C(r+b, r)$, which counts all lineups and ignores the adjacency restriction.
Returning zero only when $r > b$ instead of the correct $r > b+1$ condition.
Precomputing only through b , then indexing $\text{fact}[b+1]$.
Dividing factorials with $/$ after taking a modulo instead of multiplying by inverse factorials.

6.13 Tests that matter

$r=0, b=0$: the empty lineup is one valid lineup.
 $r=1, b=0$: exactly one lineup; $r=2, b=0$: none.
 $r=b+1$: every gap must contain one red.
 $r > b+1$: impossible without any factorial formula.
 $r=1, b=1,000,000$: checks the $b+1$ preprocessing boundary.
200,000 impossible queries: checks input/output throughput.

7 Testing and stress discussion

For A, compare small values with `math.comb`. For B, enumerate permutations of short strings and compare the number of unique strings. For C, enumerate small nonnegative vectors recursively, then compare with the formula. For D, enumerate all labeled orders for at most eight people and reject adjacent-red orders directly. Important fixed tests are:

```
A: k=0, k=n, and the largest n
B: all equal letters and all distinct letters
C: s=0, n=1, and a query that determines the global maximum n+s-1
D: r=0, r=b+1, r>b+1, and b=1,000,000
```

Run the supplied checks with:

```
CP_TARGET=solution python3 sections/32_combinatorics_under_modulo/check.py
```

8 What to carry forward

First translate a story into a mathematical object. Then determine the largest factorial index *before* coding. Prime-modulus factorial preprocessing is a powerful constant-time query engine, but only while every relevant index remains strictly below the modulus.